

ML-Based Flood Prediction System Using Historical Threat Mapping and Live Weather APIs

Submitted to: Dr. Jeelani

Amit Bishnoi (1024240005)

Diksha Garg (1024240015)

Thapar Institute of Engineering and Technology

August 2026

1 Introduction & Problem Statement

Floods cause massive damage around the world. Because of climate change, they are happening more often, making it crucial to have better ways to predict them. Right now, weather apps show us live data like rain and wind, but they do not tell us if that rain will actually cause a flood in a specific area.

On the other hand, traditional flood maps are usually drawn once and rarely updated. They cannot react to a sudden, heavy storm. Also, databases that track past disasters (like EM-DAT) have great details on the damage caused, but they often lack exact GPS coordinates. Because of this, current systems struggle to instantly connect a live storm with a location's history of flooding.

Our project aims to fix this gap. We are building a system that takes live weather data and compares it against a combined database of past floods. By merging the exact GPS data from the Dartmouth Flood Observatory (DFO) with the impact records from EM-DAT, our machine learning model will be able to predict the real-time flood risk for any searched location.

2 Literature Review

In the past, predicting floods required complex physical models of rivers and terrain. These models need a lot of computing power and local data, which is hard to get for many smaller cities and towns.

Recently, Machine Learning (ML) has become a popular and easier alternative. Many researchers have successfully used Neural Networks to predict water levels based on historical sensor data. However, there is still a gap: most systems do not combine live weather API data with long-term disaster records. Often, a location's vulnerability is treated as a fixed map rather than something that changes with the weather. This project bridges that gap by mixing real-time weather conditions with decades of historical data to give an instant, automated risk score.

3 Objectives

The main goal of this project is to build a machine learning system that uses live weather data and past disaster records to predict current flood risks.

Specific steps to achieve this include:

- To pull real-time weather data (like rainfall, temperature, and wind speed) using the OpenWeatherMap API.
- To clean and combine the EM-DAT and DFO datasets to create a complete history of floods with exact GPS coordinates.
- To create custom data features for our model, such as past flood severity and how much it rained in the last 48 hours.
- To train and test machine learning models (XGBoost, Random Forest, and LSTM) to predict if the flood risk is Low, Medium, or High.
- To build a simple user interface where anyone can search a location and see its live weather and risk score.

4 Methodology & System Architecture

4.1 Data Processing

First, we need to combine our historical data. EM-DAT has great details on human impact, while DFO provides exact GPS locations. We will write Python scripts using the Pandas library to match events between the two datasets based on the country, region, and the date the flood happened.

4.2 Feature Engineering

Once the data is combined, we will create new inputs (features) for our model to learn from. These include:

- **Magnitude Score:** How severe past floods were at that specific location.
- **Recent Rainfall:** The total rainfall over the last 48 hours, retrieved from the weather API, to check if the ground is already soaked.
- **Distance Index:** The distance from the user's searched location to the nearest historical flood point.

4.3 Machine Learning Algorithms

We will test two main types of models:

1. **XGBoost & Random Forest:** These models are great at finding patterns in structured data. They will look at the current weather and the location's history to classify the risk level.
2. **LSTM Networks:** Long Short-Term Memory models are good at looking at data over time. We will use them to analyze weather trends over a few days to see if a flood is slowly building up.

4.4 System Architecture

The project will be coded entirely in Python. We will build the backend logic using FastAPI or Flask to handle the data and the ML model. The front-end user interface will be created using Tkinter for early testing, and later Streamlit so we can view the maps and scores in a web browser.

5 Evaluation Criteria

To make sure our model actually works, we will test it using 20% of our historical data that the model has not seen before. We will measure its success using:

- **Precision, Recall, and F1-Score:** This helps us balance false alarms (saying there is a flood when there isn't) with missed warnings (missing a real flood).
- **ROC-AUC:** This checks how well the model can separate Low, Medium, and High risk levels.
- **Speed:** The system should show the final risk score in under 2 seconds after the user searches a location.

6 Project Timeline

Phase	Task Description	Key Deliverable
Weeks 1–3	Read existing research, set up API keys, and clean the data.	Cleaned dataset & working API code
Weeks 4–7	Combine DFO and EM-DAT data, and start training ML models.	Trained models & accuracy reports
Weeks 8–10	Connect the backend code to the front-end user interface.	Working software prototype
Weeks 11–12	Fix bugs, improve the design, and write the final report.	Final project file & presentation

7 Resources & Risk Assessment

7.1 Resources Needed

We will use free, open-source Python libraries (like Scikit-Learn and Pandas) and public datasets (EM-DAT and DFO). The live weather data will come from the free version of the OpenWeatherMap API. We will use our own laptops for coding and training the models, so the project requires zero budget.

7.2 Possible Risks & Solutions

- **Mismatched Data:** When combining EM-DAT and DFO, some dates or locations might not match perfectly. *Solution:* We will write code that matches events that happened within a few days of each other, rather than looking for exact date matches.

- **Imbalanced Data:** The dataset will naturally have more "safe" days than "flood" days, which can confuse the model. *Solution:* We will use a technique called SMOTE to balance the training data so the model learns fairly.
- **API Limits:** Free weather APIs only let you make a limited number of requests per day. *Solution:* We will save (cache) our recent searches so we do not have to keep calling the API for the same location while we are testing the code.